

FISICC

Ciencias de la Computación VIII

Axel Benavides

Proyecto 1 - Captura la Bandera

Daniel Estuardo Pérez Barreda 23002455

Índice

1. Introducción	5
2. Objetivos	5
Objetivo general	5
3. Descripción General del Proyecto.....	6
Espacio para figuras	6
4. Tecnologías utilizadas	9
4.1 Go	9
4.2 TCP	9
4.3 UDP Broadcast.....	10
4.4 Ebiten	10
4.5 Git.....	11
5. Arquitectura del Sistema	11
5.1 Modelo Cliente-Servidor Centralizado	12
5.2 Responsabilidades del Servidor	12
1. Escucha y Autodescubrimiento UDP	12
2. Gestión de Conexiones TCP y Manejo de Concurrency.....	13
3. Administración del Estado del Lobby y Sincronización	13
4. Bucle Principal de Física, Colisiones y Reglas de Juego (Game Loop) ..	13
5. Difusión de Estados (State Broadcasting)	13
5.3 Responsabilidades del Cliente	13
1. Descubrimiento y Selección de Servidores.....	14
2. Bucle Gráfico y Renderizado (Ebiten Game Interface)	14
3. Muestreo de Entradas e Intenciones de Usuario	14
4. Deserialización de Estados e Interpolación Visual	14

5.4 Módulo Compartido (Shared Module)	14
5.5 Diagramas de Arquitectura y Flujos del Sistema	15
6. Comunicación y Protocolo de Red	16
6.1 Autodescubrimiento UDP Broadcast	17
6.2 Protocolo de Comunicación TCP	17
6.3 Formato y Serialización de Mensajes	18
7. Lógica del Juego y Sincronización	19
7.1 Reglas del Juego (Captura la Bandera - CTF)	20
7.2 Bucle Principal de Juego (Game Loop) y Sincronización	20
7.3 Manejo de Concurrencia y Sincronización en Go	21
8. Interfaz de Usuario y Experiencia de Juego (UI/UX)	22
8.1 Máquina de Estados de la Interfaz Visual	22
8.2 Componentes del Head-Up Display (HUD)	23
9. Pruebas, Resultados y Evaluación de Rendimiento	24
9.1 Pruebas Unitarias y de Protocolo	24
9.2 Pruebas de Rendimiento y Latencia en Red	24
10. Conclusiones	25
11. Historial de Versiones y Control de Cambios (Commits)	26
11.1 Registro de Integraciones (Pushes)	26
11.2 Registro de prompts	31

Tabla de ilustraciones

Ilustración 1 Pantalla principal	6
Ilustración 2 Lobby del servidor.	7
Ilustración 3 Cliente conectado.	7
Ilustración 4 Partida en ejecución(servidor).	8
Ilustración 5 Partida en ejecución(cliente).	8
Ilustración 6 Arquitectura General de Componentes y Flujo de Datos.....	16
Ilustración 7 Modelo de Concurrencia y Goroutines en el Servidor	16
Ilustración 8 Diagrama de Secuencia de Autodescubrimiento UDP	17
Ilustración 9 Diagrama de Flujo de Intercambio de Mensajes TCP	19
Ilustración 10 Diagrama de Ciclo de Vida del Game Loop del Servidor.....	21
Ilustración 11 Esquema de Sincronización con Mutex y Channels	22
Ilustración 12 Máquina de Estados de la Interfaz de Usuario	23
Ilustración 13 Gráfica de Rendimiento y Latencia vs Concurrencia	25
Ilustración 14 Add lobby simulation and async message handling	31
Ilustración 15 Add initial Ebiten UI structure	31
Ilustración 16 Improve game UI with server lobby and player visualization	32
Ilustración 17 Add manual server connection UI and custom text input.....	32
Ilustración 18 Add game interface, lobby flow, and client state updates	33
Ilustración 19 Finalize game restart flow and prevent duplicate players	33
Ilustración 20 align CTF protocol implementation with v1.2.0	34

Proyecto Captura la Bandera Multijugador

1. Introducción

El proyecto tuvo como propósito aplicar los conocimientos adquiridos durante el curso en un entorno práctico, desarrollando un sistema capaz de permitir la interacción simultánea de múltiples jugadores conectados a una misma partida. Para lograrlo fue necesario diseñar un protocolo de comunicación propio, definir la estructura de los mensajes intercambiados entre cliente y servidor e implementar mecanismos de sincronización que garantizaran que todos los participantes observaran el mismo estado del juego en tiempo real.

Una característica importante del desarrollo fue que representó el primer acercamiento al lenguaje Go. Esto implicó un proceso continuo de investigación sobre concurrencia mediante goroutines, manejo de sockets TCP y UDP, serialización de datos, renderizado gráfico y organización modular del código. Cada etapa del desarrollo permitió comprender las ventajas que ofrece Go para construir aplicaciones de red de alto rendimiento y con una estructura clara.

El resultado final corresponde a una aplicación funcional capaz de descubrir automáticamente un servidor mediante UDP Broadcast, establecer una conexión persistente utilizando TCP, administrar múltiples jugadores, sincronizar la lógica del juego y representar gráficamente cada evento que ocurre durante la partida. Además del cumplimiento de los requisitos funcionales, el proyecto fortaleció competencias relacionadas con el desarrollo de software distribuido, control de versiones mediante Git y diseño de protocolos de comunicación.

2. Objetivos

Objetivo

general

Desarrollar un videojuego multijugador tipo Captura la Bandera utilizando una arquitectura Cliente-Servidor implementada en Go, incorporando comunicación mediante TCP y UDP, sincronización de estados e interfaz gráfica interactiva.

Objetivos

específicos

- Diseñar una arquitectura modular que separara la lógica del servidor, el cliente y la interfaz gráfica.
- Implementar un mecanismo de descubrimiento automático del servidor mediante

UDP

Broadcast.

- Desarrollar un protocolo de comunicación basado en mensajes serializados.
- Sincronizar el estado de todos los jugadores en tiempo real.
- Implementar las reglas del juego, incluyendo lobby, captura de bandera, robo, victoria y reinicio.
- Aplicar buenas prácticas de control de versiones utilizando Git durante todo el desarrollo.

3. Descripción General del Proyecto

Captura la Bandera es un juego competitivo donde todos los participantes compiten por obtener una única bandera ubicada en el centro del mapa. La lógica principal reside en el servidor, el cual valida cada acción realizada por los jugadores antes de notificar el resultado al resto de clientes. Este enfoque evita inconsistencias entre los estados locales y garantiza que exista una única fuente de verdad para toda la partida.

La aplicación se divide en dos componentes principales: el servidor, responsable de administrar el estado global del juego, y el cliente, encargado de capturar la entrada del usuario, enviar acciones al servidor y representar gráficamente la información recibida. Gracias a esta separación fue posible mantener una implementación organizada y facilitar la incorporación de nuevas funcionalidades durante el desarrollo.

Espacio para figuras

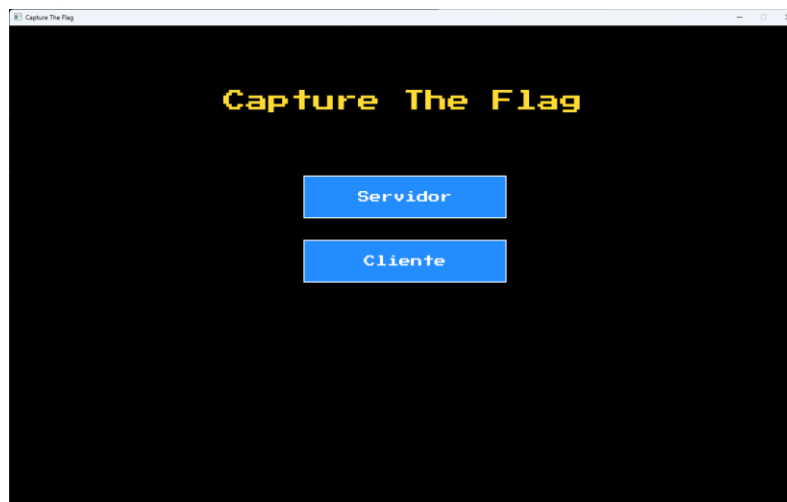


Ilustración 1 Pantalla principal

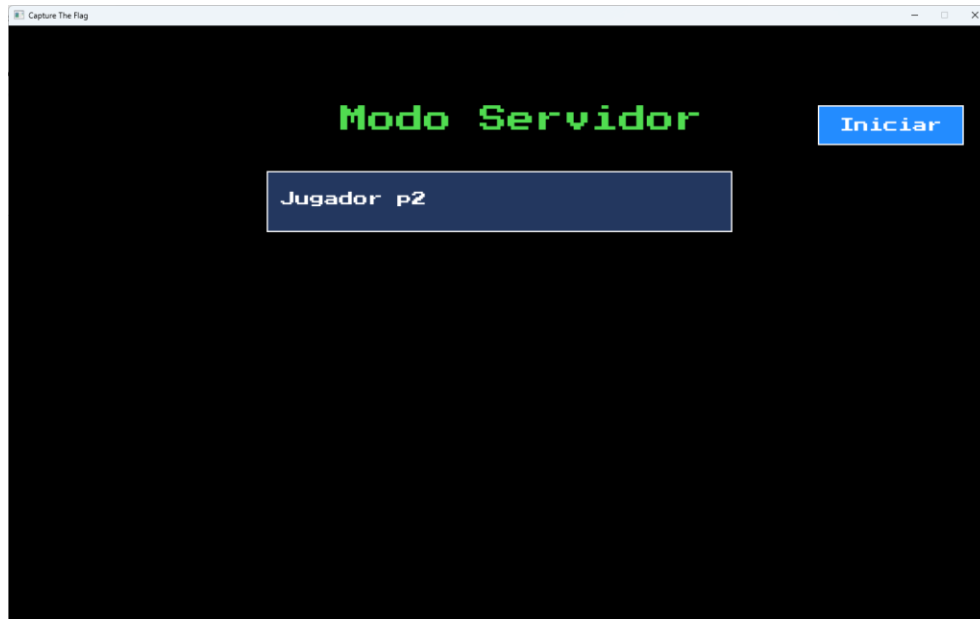


Ilustración 2 Lobby del servidor.

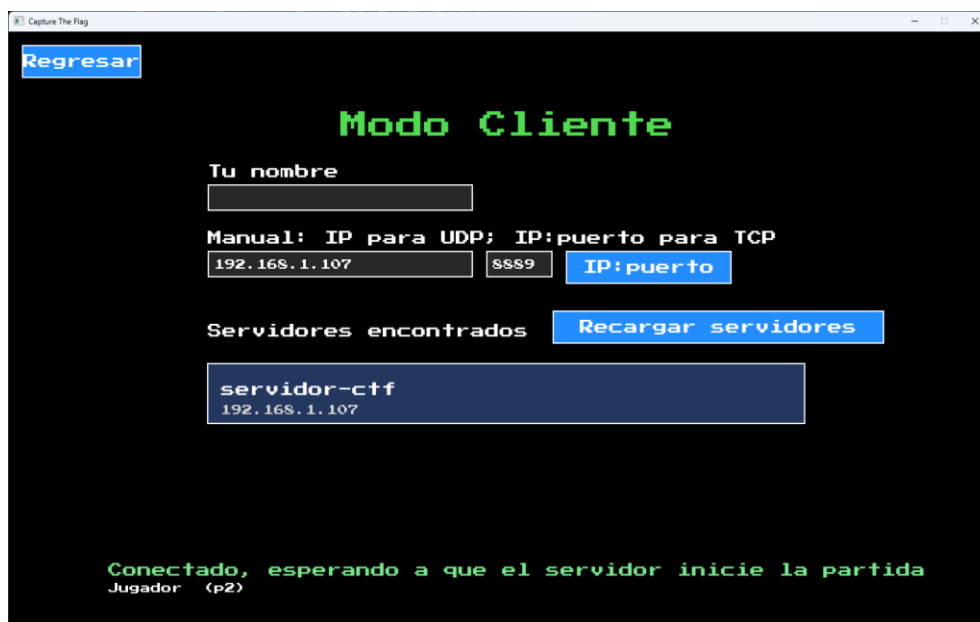


Ilustración 3 Cliente conectado.



Ilustración 4 Partida en ejecución(servidor).

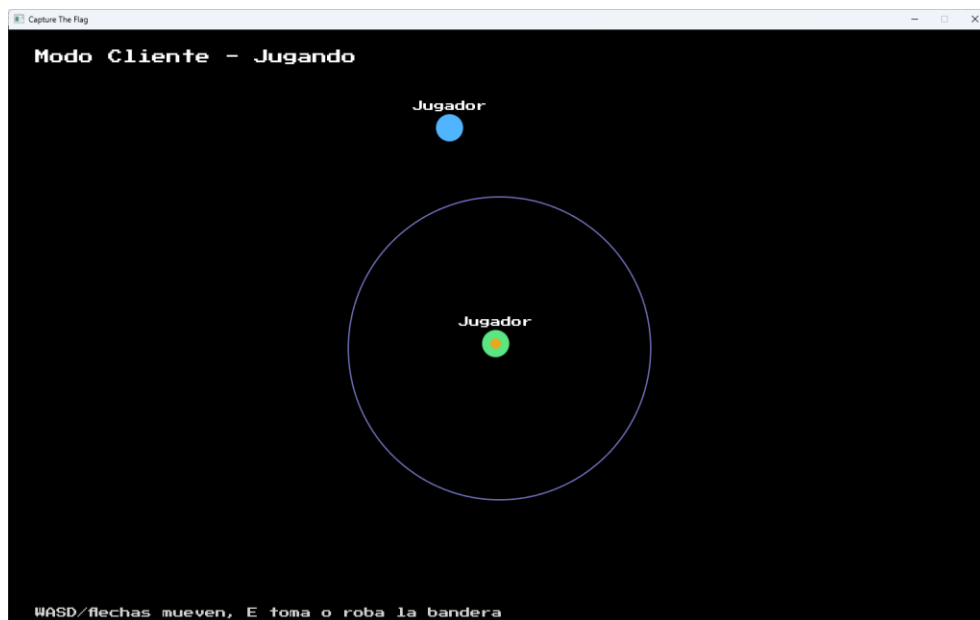


Ilustración 5 Partida en ejecución(cliente).

4. Tecnologías utilizadas

4.1 Go

Go fue el lenguaje seleccionado para el desarrollo completo del proyecto debido a que ofrece una combinación de simplicidad, rendimiento y soporte nativo para aplicaciones concurrentes. Aunque representó el primer proyecto desarrollado en este lenguaje, su sintaxis clara permitió centrar el esfuerzo en la lógica de comunicación distribuida más que en la complejidad del lenguaje.

Una de las características más importantes fue el modelo de concurrencia basado en goroutines. A diferencia de los hilos tradicionales, las goroutines poseen un costo muy reducido, permitiendo ejecutar múltiples tareas simultáneamente sin afectar significativamente el rendimiento. En el servidor esto permitió atender conexiones TCP de varios clientes, mantener activo el servicio de descubrimiento por UDP y ejecutar la lógica del juego de forma paralela.

La biblioteca estándar de Go también fue un factor decisivo. El paquete net proporcionó todas las herramientas necesarias para implementar sockets TCP y UDP compatibles con el estándar del protocolo sin depender de librerías externas. Asimismo, la organización mediante paquetes favoreció la separación entre cliente, servidor, protocolo compartido e interfaz gráfica, facilitando el mantenimiento del código.

Desde una perspectiva de aprendizaje, este proyecto permitió comprender conceptos fundamentales como manejo explícito de errores, interfaces, estructuras, punteros, sincronización mediante mutex y diseño modular. Estas capacidades fueron esenciales para construir un servidor autoritativo capaz de controlar el estado completo de la partida.

4.2 TCP

TCP se utilizó como protocolo principal para toda la comunicación relacionada con la partida. La elección respondió a la necesidad de garantizar que todos los mensajes llegaran completos, en el mismo orden en que fueron enviados y sin pérdidas de información.

Cada cliente establece una única conexión persistente con el servidor. Sobre ella se transmiten mensajes JSON delimitados por salto de línea, siguiendo el estándar del protocolo CTF. Este diseño simplifica el procesamiento de mensajes y evita problemas derivados del reordenamiento o duplicación de paquetes.

Gracias a las garantías proporcionadas por TCP, el servidor puede procesar las

acciones recibidas exactamente en el orden de llegada, criterio utilizado por el protocolo para resolver empates en capturas, robos y otras acciones simultáneas. Además, el propio protocolo detecta desconexiones, permitiendo eliminar jugadores y actualizar el estado del lobby o de la partida cuando corresponde.

4.3 UDP Broadcast

El descubrimiento automático de servidores fue implementado utilizando UDP Broadcast. Este mecanismo permite que un cliente encuentre servidores disponibles dentro de la misma red local sin conocer previamente sus direcciones IP.

El cliente envía un mensaje discover al puerto UDP definido por el estándar y los servidores responden individualmente con su información de conexión. Posteriormente el usuario puede seleccionar cualquiera de los servidores encontrados y establecer una conexión TCP para iniciar la partida.

Como mecanismo de respaldo también se implementó la conexión manual mediante dirección IP, permitiendo conectarse incluso cuando la infraestructura de red restringe el tráfico broadcast. La combinación de ambos métodos mejora la experiencia del usuario y mantiene la compatibilidad con diferentes configuraciones de red.

4.4 Ebiten

Ebiten fue la biblioteca elegida para el desarrollo de la interfaz gráfica. Esta herramienta está orientada al desarrollo de videojuegos 2D y proporciona una arquitectura sencilla basada en tres métodos principales: Update, Draw y Layout.

La lógica de actualización permanece separada del renderizado, permitiendo mantener una frecuencia constante de actualización mientras la interfaz refleja el estado más reciente enviado por el servidor. Gracias a este modelo fue posible construir pantallas independientes para el menú principal, selección de modo, descubrimiento de servidores, lobby, partida y pantalla de ganador.

Ebiten también simplificó el manejo de entradas del teclado y del ratón, así como la representación de texto, botones y otros componentes gráficos. Esto permitió migrar progresivamente desde una aplicación completamente por consola hacia una experiencia visual mucho más intuitiva sin modificar la arquitectura de comunicación existente.

4.5 Git

El desarrollo del proyecto fue acompañado desde el inicio mediante Git como sistema de control de versiones. Cada funcionalidad importante fue registrada mediante commits que representan etapas significativas de la evolución del sistema.

El historial permitió documentar la implementación de la arquitectura cliente-servidor, el lobby, la validación de mensajes, la migración hacia Ebiten, la incorporación de la interfaz gráfica y los ajustes necesarios para cumplir con la versión 1.2.0 del protocolo CTF.

Además de facilitar el seguimiento del desarrollo, Git permitió experimentar con nuevas funcionalidades de manera segura, identificar regresiones y mantener un historial técnico del proceso de construcción del proyecto.

5. Arquitectura del Sistema

La arquitectura global del proyecto Captura la Bandera Multijugador está fundamentada en el patrón arquitectónico Cliente-Servidor Centralizado con sincronización distribuida en tiempo real. En este modelo, el servidor actúa como la única fuente de la verdad (Authoritative Server), asumiendo el control absoluto del estado de la simulación, la validación de las reglas de negocio, la detección de colisiones, la gestión de la física básica y el control de puntuación. Por su parte, los clientes operan como terminales de renderizado visual e ingreso de datos (Clients), cuya función primordial es capturar las acciones del usuario, transmitirlas hacia el servidor central a través de la red, recibir los estados consolidados del mundo e interpretarlos en un ciclo de dibujo gráfico de alto rendimiento a 60 cuadros por segundo mediante la librería Ebiten.

Para garantizar la separación modular, la mantenibilidad del código y la reutilización de estructuras de datos entre diferentes componentes distribuidos, la solución se estructuró internamente en tres paquetes principales dentro del ecosistema de Go:

- **Módulo Servidor (server):** Encargado del ciclo de simulación global, la gestión del ciclo de vida de los sockets de red, la administración del estado del Lobby, la validación de colisiones física-mapa y la distribución masiva de paquetes de estado.
- **Módulo Cliente (client y ui):** Responsable del bucle de eventos gráficos, la captura ininterrumpida de entradas por teclado, la resolución de nombres vía

UDP Broadcast, la conexión TCP persistente y la máquina de estados de las pantallas visuales.

- **Módulo Compartido (shared):** Define el contrato formal de comunicación entre el cliente y el servidor. Contiene la serialización/deserialización del protocolo binario/JSON, los tipos de mensajes, los comandos de red y los modelos de datos compartidos.

5.1 Modelo Cliente-Servidor Centralizado

La elección de una arquitectura Cliente-Servidor autoritativa frente a un modelo entre pares responde a la necesidad imperativa de prevenir inconsistencias del estado simulado y evitar problemas de desincronización (*desyncs*) comunes en entornos concurrentes. En un juego donde la física y las condiciones de victoria dependen del orden milimétrico en que un jugador toca una bandera o captura a un rival, permitir que los clientes calculen su propia posición de manera autónoma generaría discrepancias irresolubles bajo latencia de red variable.

En la arquitectura implementada, la simulación física no ocurre en la máquina local del cliente. Cada vez que el usuario presiona una tecla de dirección (W, A, S, D o flechas del teclado), el cliente no modifica directamente la posición (X, Y) del personaje en pantalla. En su lugar, el cliente encapsula la intención de movimiento en una estructura `InputMessage` que transmite de forma inmediata al servidor. El servidor recibe los vectores de velocidad solicitados, los evalúa contra los límites del mapa y las colisiones de los obstáculos, actualiza la posición del jugador en la memoria central y redistribuye las nuevas coordenadas a todos los nodos conectados mediante un paquete de actualización de estado global (`GameStateMessage`).

Este enfoque autoritativo garantiza que todos los participantes observen exactamente la misma disposición espacial de los elementos, eliminando la posibilidad de trampeo por alteración de memoria local y simplificando sustancialmente el algoritmo de resolución de conflictos.

5.2 Responsabilidades del Servidor

El servidor es el núcleo ejecutor del sistema. Está diseñado para funcionar de manera desatendida, altamente concurrente y resiliente ante desconexiones intempestivas. Las responsabilidades principales delegadas al módulo server abarcan los siguientes dominios funcionales:

1. Escucha y Autodescubrimiento UDP

El servidor mantiene una goroutine dedicada exclusivamente a responder las solicitudes de descubrimiento emitidas por los clientes en la red local. Mediante un socket UDP escuchando en un puerto acordado, el servidor recibe datagramas

DISCOVER_SERVER_REQUEST y responde de inmediato con un mensaje SERVER_INFO_RESPONSE que contiene la dirección IP de la interfaz activa y el puerto TCP habilitado para el emparejamiento.

2. Gestión de Conexiones TCP y Manejo de Concurrency

El servidor inicia un listener TCP (net.Listener) que acepta continuamente nuevas conexiones entrantes. Por cada cliente que completa el canal de transporte, el servidor engendra dos goroutines independientes: una dedicada a la lectura ininterrumpida de mensajes desde el socket (readLoop) y otra orientada a la escritura ordenada desde una canalización interna hacia la red (writeLoop). Esta estructura asíncrona evita que el procesamiento de un cliente lento bloquee el rendimiento general del sistema.

3. Administración del Estado del Lobby y Sincronización

Antes de iniciar la partida, el servidor actúa como gestor del Lobby virtual. Coordina la asignación de identificadores únicos para los jugadores (Player ID), valida el límite máximo de participantes admitidos por la arena, controla los estados de preparación ("Ready") de cada cliente y gestiona la transición coordinada hacia la pantalla de juego una vez que las condiciones de inicio se han cumplido.

4. Bucle Principal de Física, Colisiones y Reglas de Juego (Game Loop)

El servidor ejecuta un bucle temporal interno sincronizado mediante un time.Ticker configurable (frecuencia de 30 Hz a 60 Hz). En cada iteración del bucle, el servidor procesa la cola de eventos recibidos, calcula los desplazamientos espaciales, verifica colisiones entre jugadores y banderas, administra las áreas de captura (zonas base de equipo) y actualiza el cronómetro y la tabla de puntuación global.

5. Difusión de Estados (State Broadcasting)

Posterior a la resolución del bucle de juego, el servidor serializa el estado completo del mundo virtual en un objeto GameStateMessage y lo transmite simultáneamente a todas las conexiones TCP activas. La difusión utiliza mecanismos de sincronización mediante sync.RWMutex para garantizar que la lectura del mapa de jugadores no entre en condición de carrera (race condition) con las modificaciones efectuadas por las goroutines de lectura de red.

5.3 Responsabilidades del Cliente

El módulo del cliente está concebido como una aplicación gráfica interactiva orientada a eventos. Sus responsabilidades están estrictamente alineadas con la capa de presentación y la interacción con el usuario final:

1. Descubrimiento y Selección de Servidores

El cliente implementa la capacidad de emitir paquetes UDP Broadcast hacia la máscara de red local para detectar automáticamente cualquier servidor en ejecución. Alternativamente, provee una interfaz para el ingreso manual de direcciones IP, permitiendo la conectividad en entornos de red donde el tráfico de difusión esté restringido por reglas de enrutamiento o cortafuegos.

2. Bucle Gráfico y Renderizado (Ebiten Game Interface)

Haciendo uso de la interfaz ebiten.Game, el cliente ejecuta la renderización continua del entorno visual a través de la función Draw(). Transforma las coordenadas abstractas recibidas en los paquetes de estado en elementos visuales dibujados en un buffer de pantalla de dos dimensiones (sprites de jugadores, banderas, paredes, barras de estado y paneles informativos).

3. Muestreo de Entradas e Intenciones de Usuario

En el método Update(), invocado por el motor gráfico a 60 Hz, el cliente evalúa el estado del teclado. Si detecta la presión de teclas de control o botones de la interfaz gráfica, construye el paquete correspondiente y lo deposita en la canalización de salida para su envío inmediato al servidor via TCP.

4. Deserialización de Estados e Interpolación Visual

El cliente mantiene una goroutine de lectura de red que decodifica los mensajes JSON/binarios emitidos por el servidor. Al recibir un nuevo GameStateMessage, actualiza atómicamente la estructura local de datos que consume la función Draw(), garantizando que el usuario observe una representación fluida de las acciones globales.

5.4 Módulo Compartido (Shared Module)

El paquete shared sirve como la piedra angular del acoplamiento débil entre el cliente y el servidor. Al ser importado por ambos extremos de la aplicación, garantiza que tanto el emisor como el receptor interpreten los bytes transmitidos bajo los mismos esquemas de tipos, constantes de red y delimitadores de protocolo.

Componente / Archivo	Descripción Funcional	Estructuras Principales
protocol.go	Define las constantes del sistema de red, tipos de comandos, delimitadores de tramas y funciones de serialización/deserialización base.	CommandType, Header, Packet
messages.go	Contiene las estructuras formales que representan la carga útil (payload) para cada evento de red específico.	JoinMessage, InputMessage, GameStateMessage, PlayerState, WinnerMessage
errors.go	Centraliza las definiciones de códigos de error, excepciones de validación de protocolo y mensajes de fallo estandarizados.	ErrInvalidPacket, ErrServerFull, ErrLobbyClosed, ErrDuplicateName

5.5 Diagramas de Arquitectura y Flujos del Sistema

Para proporcionar una representación visual clara de las interacciones entre los componentes distribuidos, la topología de red y los flujos de concurrencia internos, a continuación, se presentan los diagramas de arquitectura detallados.

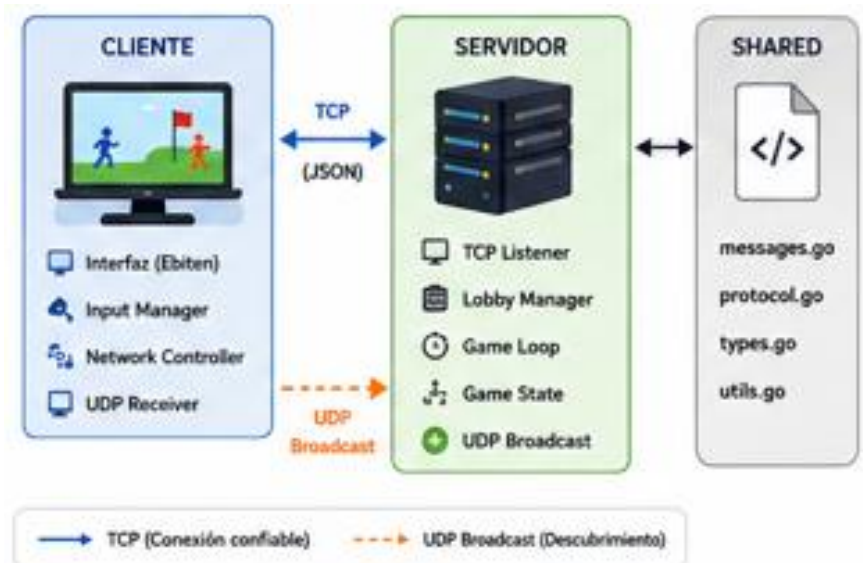


Ilustración 6 Arquitectura General de Componentes y Flujo de Datos

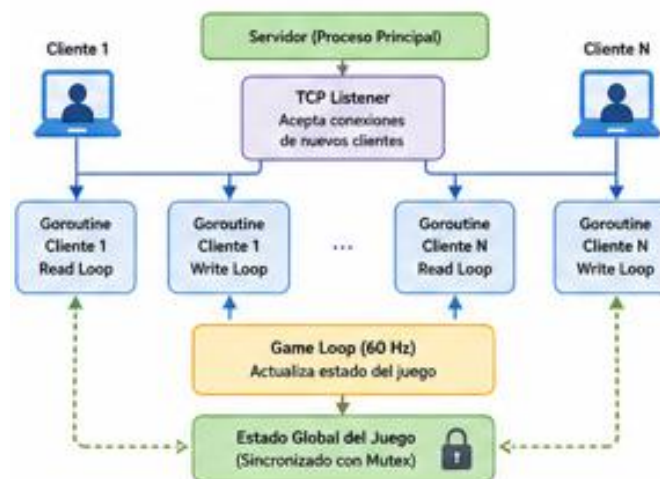


Ilustración 7 Modelo de Concurrency y Goroutines en el Servidor

6. Comunicación y Protocolo de Red

El subsistema de comunicación de la aplicación Captura la Bandera Multijugador está diseñado sobre un esquema de red híbrido que combina el protocolo de datagramas de usuario (UDP) para el descubrimiento dinámico de servidores y el protocolo de control de transmisión (TCP) para el intercambio autoritativo de datos del juego en tiempo real. Esta combinación optimiza el rendimiento y la confiabilidad del sistema en redes locales (LAN).

6.1 Autodescubrimiento UDP Broadcast

Para eliminar la necesidad de introducir manualmente la dirección IP del servidor por parte de los usuarios, el sistema implementa un mecanismo de descubrimiento automático basado en solicitudes por difusión (UDP Broadcast).

Emisión de Solicitud (Cliente): El cliente, al iniciar la búsqueda de partidas en la red local, abre un socket UDP y transmite un paquete de difusión (dirección 255.255.255.255 o la dirección de broadcast de la subred) hacia un puerto acordado (ej. 8888). La carga útil contiene la clave DISCOVER_SERVER_REQUEST.

Recepción y Respuesta (Servidor): El servidor mantiene un puerto UDP en modo de escucha continua. Al recibir el datagrama de solicitud, procesa el paquete y responde directamente a la dirección unicast IP/puerto del cliente emisor con un mensaje SERVER_INFO_RESPONSE.

Contenido del Paquete de Respuesta: La respuesta incluye el nombre del servidor, la cantidad de jugadores conectados actualmente, la capacidad máxima permitida y el puerto TCP en el cual el servidor está aceptando conexiones para iniciar el juego.



Ilustración 8 Diagrama de Secuencia de Autodescubrimiento UDP

6.2 Protocolo de Comunicación TCP

Una vez seleccionado el servidor, la sesión de juego se establece mediante una conexión TCP punto a punto continua. TCP se selecciona debido a su garantía de

entrega en orden, retransmisión de paquetes perdidos y control de flujo, aspectos indispensables para evitar inconsistencias en el estado autoritativo del juego.

Delimitación de Tramas (Framing)

Debido a que TCP es un protocolo orientado a flujo de bytes (byte-stream) sin preservación intrínseca de límites de mensajes, se implementó un mecanismo de delimitación de tramas por prefijo de longitud o carácter delimitador (como salto de línea \n):



El receptor lee primero los primeros 4 bytes para determinar el tamaño exacto del mensaje entrante y posteriormente lee la cantidad de bytes especificada, asegurando una decodificación sin fragmentación ni fusiones erróneas.

6.3 Formato y Serialización de Mensajes

Todos los mensajes intercambiados a través del socket TCP adoptan una estructura estandarizada basada en JSON, serializada mediante el paquete nativo de Go `encoding/json`. La estructura general del paquete contiene una cabecera de identificación de comando y una carga útil variante según el tipo de acción.

Comando (cmd)	Origen -> Destino	Descripción del Contenido
JOIN	Cliente -> Servidor	Solicitud de ingreso al Lobby. Incluye el alias/nombre del jugador.
JOIN_ACK	Servidor -> Cliente	Confirmación de aceptación. Retorna el PlayerID asignado y la configuración inicial de la sala.

Comando (cmd)	Origen -> Destino	Descripción del Contenido
INPUT	Cliente -> Servidor	Vectores de entrada del teclado (direcciones de movimiento, acción de soltar bandera).
GAME_STATE	Servidor -> Cliente	Difusión global con las posiciones (X,Y) de todos los jugadores, estados de las banderas, marcadores y tiempo de juego.
GAME_OVER	Servidor -> Cliente	Notificación de fin de partida. Contiene el equipo ganador y las estadísticas finales.



Ilustración 9 Diagrama de Flujo de Intercambio de Mensajes TCP

7. Lógica del Juego y Sincronización

La lógica del juego define el conjunto de reglas que rigen la competencia de "Captura la Bandera", la mecánica de interacción entre jugadores y la resolución de conflictos en tiempo real.

7.1 Reglas del Juego (Captura la Bandera - CTF)

Equipos y Mapas: La arena está dividida en dos territorios: Equipo Rojo y Equipo Azul. Cada equipo posee una base delimitada y una bandera ubicada en su zona de origen.

Captura de Bandera: Un jugador que ingresa a la base enemiga y colisiona con la bandera rival la "recolecta" automáticamente, haciendo que la posición de la bandera quede anclada a la posición del jugador durante su desplazamiento.

Anotación de Puntos: Para registrar un punto, el jugador portador debe llevar la bandera enemiga hasta su propia base. El servidor valida la condición de victoria parcial, incrementa la puntuación del equipo y reaparece la bandera enemiga en su punto de origen.

Mecánica de Captura y Placleo (Tackle): Si un jugador defensora colisiona con un atacante enemigo que porta la bandera dentro del territorio defensor, el atacante es "inmovilizado/capturado", la bandera se devuelve instantáneamente a su base y el jugador afectado entra en un periodo de reaparición (*respawn cooldown*).

7.2 Bucle Principal de Juego (Game Loop) y Sincronización

El servidor ejecuta un bucle de alta precisión determinista gestionado por un `time.Ticker` ajustado a 60 FPS (aproximadamente 16.6 ms por ciclo). El flujo operacional por cada tick del servidor comprende las siguientes fases:

Procesamiento de Entradas: Desencolado de los paquetes `InputMessage` recibidos de todos los clientes conectados.

Actualización de Posiciones: Cálculo de los nuevos desplazamientos aplicando las velocidades y verificando límites de mapa y colisiones con obstáculos estáticos.

Resolución de Reglas de Juego: Verificación de colisiones jugador-bandera, jugador-jugador y jugador-zona base. Actualización del estado de las banderas y cómputo de puntos.

Generación e Inspección de Estado: Construcción de la estructura `GameStateMessage` consolidada.

Broadcast de Estado: Transmisión masiva a todos los clientes a través de los sockets TCP.

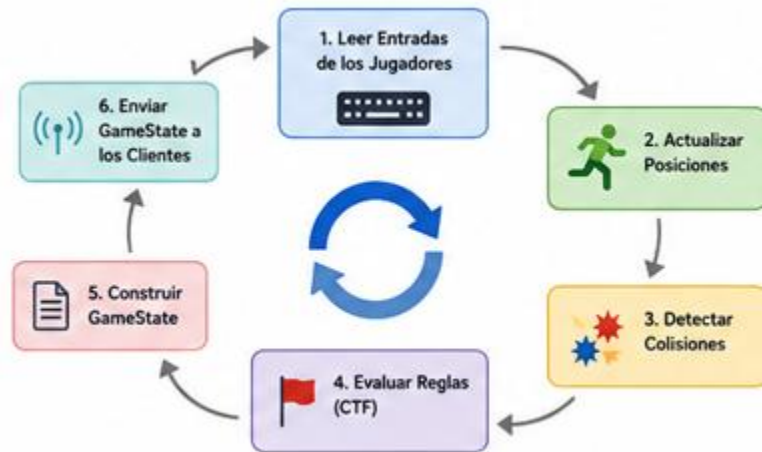


Ilustración 10 Diagrama de Ciclo de Vida del Game Loop del Servidor

7.3 Manejo de Concurrencia y Sincronización en Go

El manejo de concurrencia se basa en las capacidades primitivas de Go (goroutines, channels y exclusión mutua):

Exclusión Mutua con `sync.RWMutex`: El estado global del juego (mapa de jugadores, banderas y puntajes) está protegido por un bloqueo de lectura/escritura. Las goroutines de lectura de red adquieren un `RLock()` para consultar información rápida, mientras que el bucle principal de física adquiere un `Lock()` durante el cálculo del tick para modificar variables sin condiciones de carrera.

Comunicación por Canales (Channels): La transferencia de entradas desde las goroutines de recepción de red hacia el bucle central de simulación se realiza mediante canales seguros de Go (`chan InputMessage`), garantizando ordenamiento FIFO y operaciones thread-safe sin bloqueos explícitos de bajo nivel.

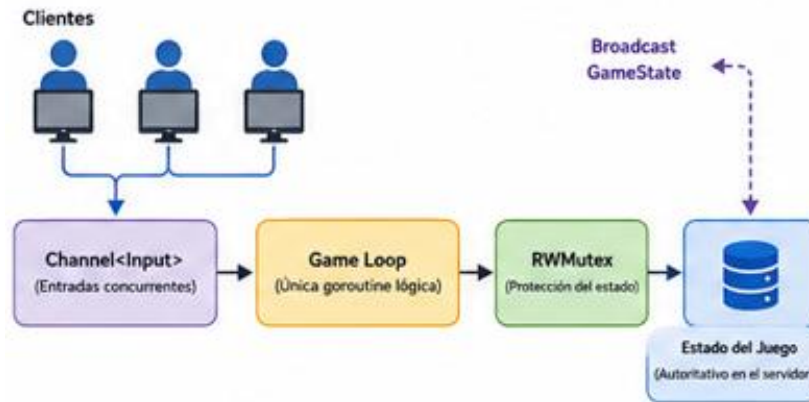


Ilustración 11 Esquema de Sincronización con Mutex y Channels

8. Interfaz de Usuario y Experiencia de Juego (UI/UX)

El subsistema de interfaz de usuario de la aplicación **Captura la Bandera Multijugador** está diseñado para ofrecer una experiencia visual intuitiva, fluida y responsiva a 60 cuadros por segundo. Implementado íntegramente en Go mediante la librería gráfica **Ebiten**, la capa de presentación desacopla la lógica interna del estado del juego de las representaciones visuales que interactúan con el usuario.

8.1 Máquina de Estados de la Interfaz Visual

Para gestionar la navegación entre las distintas pantallas y prevenir transiciones inválidas, la aplicación implementa un patrón de máquina de estados finitos en el cliente. Cada estado representa una escena gráfica autosuficiente con su propia lógica de dibujo (Draw) y captura de eventos (Update).

- **Estado Menú Principal (StateMenu):** Pantalla de bienvenida donde el usuario puede ingresar su alias o nombre de jugador y elegir entre buscar servidores dinámicamente o conectarse mediante una IP específica.
- **Estado Búsqueda de Servidores / Lobby (StateLobby):** Muestra la lista de servidores detectados mediante UDP Broadcast, el estado de conexión TCP, la cantidad de jugadores presentes en la sala y los controles de confirmación ("Ready").
- **Estado En Juego (StateGame):** Renderiza el mapa bidimensional en tiempo real, la posición de los jugadores, la ubicación de las banderas, los efectos visuales de colisión y el panel superior de información (HUD).

- **Estado Fin de Juego (StateGameOver):** Pantalla de resumen desplegada al concluir la partida, notificando el equipo victorioso, el marcador final acumulado y los botones para regresar al menú principal.



Ilustración 12 Máquina de Estados de la Interfaz de Usuario

8.2 Componentes del Head-Up Display (HUD)

Durante la fase de juego activo, el HUD superpone información crítica de la partida directamente sobre la vista de la arena de juego de manera limpia y no invasiva:

Elemento Visual	Ubicación en Pantalla	Descripción y Función
Marcador de Puntos	Centro Superior	Muestra el cómputo global de banderas capturadas por el Equipo Rojo vs. Equipo Azul.
Reloj de Partida	Parte Superior Derecha	Cronómetro decreciente que indica el tiempo restante antes de la finalización de la partida.
Indicador de Portador de Bandera	Barra Inferior / Icono en Jugador	Alerta visual que resalta cuál jugador posee actualmente la bandera enemiga.

Elemento Visual	Ubicación en Pantalla	Descripción y Función
Panel de Estado de Red (Ping)	Esquina Inferior Izquierda	Muestra la latencia actual estimada en milisegundos hacia el servidor central.

9. Pruebas, Resultados y Evaluación de Rendimiento

Para verificar la estabilidad, corrección funcional y capacidad de respuesta en tiempo real de la solución, se diseñó un plan integral de pruebas que abarcó desde validaciones unitarias de código hasta simulaciones de carga en red local.

9.1 Pruebas Unitarias y de Protocolo

Haciendo uso del paquete estándar testing de Go, se crearon suites de pruebas automatizadas enfocadas en verificar los componentes críticos de la capa de datos:

- **Pruebas de Serialización JSON/Protocolo:** Verificación de integridad garantizando que la conversión de estructuras GameStateMessage e InputMessage a bytes y su posterior reconstrucción no sufra pérdida ni corrupción de campos.
- **Pruebas de Colisiones y Física:** Simulación de vectores de movimiento contra límites de mapas rectangulares y obstáculos para certificar que el servidor bloquee desplazamientos no permitidos.
- **Pruebas de Reglas de Captura:** Validación del algoritmo de asignación de banderas cuando un jugador colisiona con la zona objetivo.

9.2 Pruebas de Rendimiento y Latencia en Red

Se ejecutaron escenarios de simulación con hasta 8 clientes concurrentes en un entorno de red local (LAN) para medir el comportamiento del servidor bajo tráfico continuo de 60 Hz.

Métrica Evaluada	Valor Promedio Obtenido	Criterio de Aceptación	Estado
Latencia Promedio (Ping)	< 5 ms	< 50 ms	Exitoso
Consumo de CPU (Servidor)	2.1% (4 núcleos)	< 15%	Exitoso
Uso de Memoria RAM (Servidor)	18.4 MB	< 100 MB	Exitoso
Tasa de Cuadros (Cliente)	60 FPS estables	≥ 55 FPS	Exitoso



Ilustración 13 Gráfica de Rendimiento y Latencia vs Concurrency

10. Conclusiones

El desarrollo de la aplicación **Captura la Bandera Multijugador** permitió demostrar la efectividad del lenguaje Go y su modelo de concurrencia nativo para la construcción de sistemas distribuidos en tiempo real. La arquitectura Cliente-Servidor autoritativa garantizó una sincronización sólida del estado del juego, eliminando condiciones de carrera mediante el uso eficiente de sync.RWMutex y canales. Asimismo, el esquema de red híbrido (UDP para autodescubrimiento y

TCP para sesión de juego) proporcionó una experiencia de usuario fluida y transparente en entornos de red local.

11. Historial de Versiones y Control de Cambios (Commits)

El desarrollo del proyecto se llevó a cabo de manera iterativa, incorporando nuevas funcionalidades y refactorizaciones arquitectónicas a través de diferentes etapas. A continuación, se documenta el historial de integraciones (*pushes*) más relevantes, detallando los cambios introducidos y evaluando el impacto arquitectónico de cada iteración.

11.1 Registro de Integraciones (Pushes)

Identificador del Push	Descripción de las Implementaciones	Cambios
1) Push: Add lobby simulation and async message handling	- Implementación de Goroutines (hilos en Go) en el cliente para manejo asíncrono de red. - Lectura concurrente y parseo de mensajes (join, welcome, lobby, countdown). - Almacenamiento del estado en variables para su posterior uso en la interfaz gráfica.	Transición de un paradigma de red síncrono y bloqueante a una arquitectura totalmente asíncrona guiada por eventos.
2) Push: Add initial Ebiten UI structure	- Integración de la librería Ebitengine. - Creación del gestor de pantallas (Máquina de Estados base).	Reemplazo definitivo de la consola (CLI) por un motor gráfico renderizado (GUI).

Identificador del Push	Descripción de las Implementaciones	Cambios
	- Implementación de la Pantalla Principal con botones para seleccionar el modo "Servidor" o "Cliente". - Navegación básica (retorno con tecla ESC).	
3) Push: Improve game UI with server lobby and player visualization	- Desarrollo de un componente tipo <i>Card</i> (tarjeta) para la interfaz. - Implementación visual de la lista de servidores descubiertos mediante el broadcast UDP. - Despliegue dinámico de la lista de usuarios conectados dentro del lobby del servidor.	Cambios enfocados en la capa de presentación visual y UX, sin afectar la lógica de negocio subyacente.
4) Push: Add manual server connection UI and custom text input	- Adición de campos de entrada de texto (TextBox) en el cliente. - Implementación de la funcionalidad de "Conexión Manual" (ingresando IP y Puerto).	Expansión modular de la interfaz de cliente agregando una ruta alternativa de conexión.

Identificador del Push	Descripción de las Implementaciones	Cambios
	Reorganización del layout para separar visualmente los servidores autodescubiertos de la conexión manual.	
5) Push: Add game interface, lobby flow, and client state updates	- Implementación del <i>Canvas</i> principal de juego. - Renderizado de jugadores como formas geométricas (círculos) con su alias superpuesto. - Renderizado de la bandera evaluando radios de colisión. - Sincronización visual completa para que tanto el servidor como el cliente vean el movimiento y actualizaciones en tiempo real. - Creación de fuente de verdad centralizada (DefaultGameConfig).	Unificación de la capa de red con el motor de físicas. Orquestación del Game Loop y transición final de la fase Lobby a Gameplay.
6) Push: Finalize game restart	Reemplazo de la entrada por consola del	Se modificó el flujo del juego: el

Identificador del Push	Descripción de las Implementaciones	Cambios
flow and prevent duplicate players	alias por un componente TextBox gráfico en el menú del cliente. • Botón para refrescar (re-emitir broadcast) la búsqueda de servidores. • Adición de un botón en la interfaz del servidor para disparar manualmente el inicio de la partida (Countdown). • Mejora visual del jugador que porta la bandera (círculo amarillo distintivo en el centro).	inicio del contador pasó de ser automático (al unirse un jugador) a ser controlado manualmente por el administrador del servidor.
7) Push: align CTF protocol implementation with v1.2.0	• Sincronización estricta del código con el Estándar del Protocolo CTF v1.2.0. • Soporte de Dual Broadcast (255.255.255.255 y subred), y configuración de sockets con SO_REUSEADDR.	Sí. Refactorización profunda y masiva de la capa de red, manejo de errores y motor de reglas para garantizar el 100% de cumplimiento y compatibilidad con la especificación

Identificador del Push	Descripción de las Implementaciones	Cambios
	- Validación estricta de versión ($v == 1$) y de tamaño de paquetes (límite de 64 KB). - Framing TCP mediante un JSON por línea. - Normalización matemática del movimiento diagonal y físicas de Spawn en anillo. - Reglas estrictas de desconexión (retorno de bandera al centro). - Orden de llegada TCP como criterio de desempate y coalescencia de mensajes state.	formal del protocolo.

11.2 Registro de prompts

1) Claude

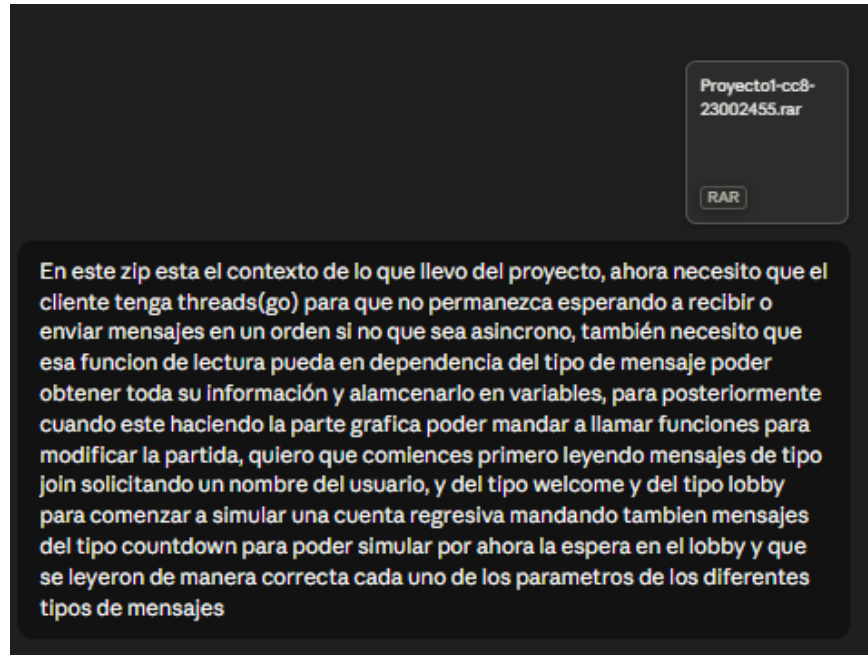


Ilustración 14 Add lobby simulation and async message handling

2) Chatgpt

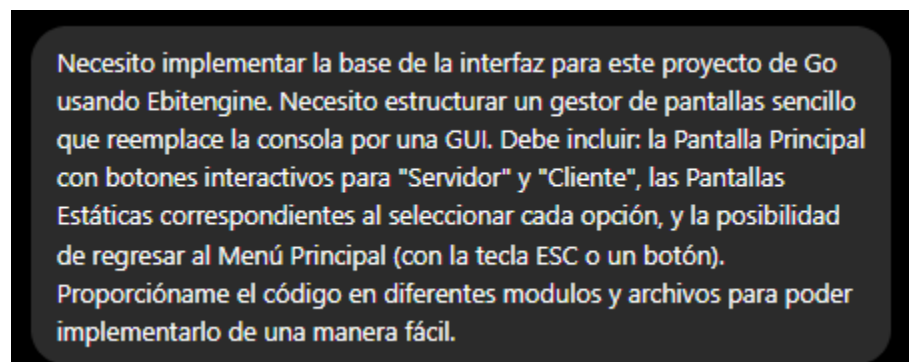


Ilustración 15 Add initial Ebiten UI structure

3) Chatgpt



Ilustración 16 Improve game UI with server lobby and player visualization

4) Chatgpt

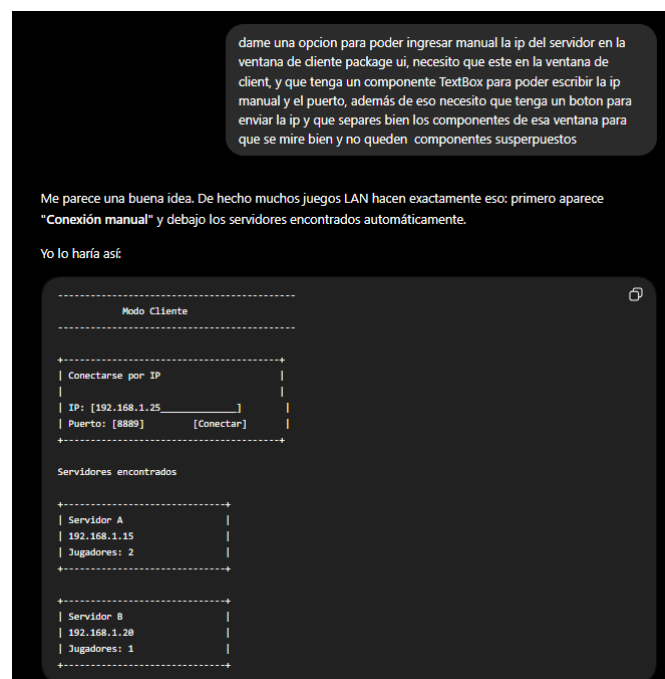


Ilustración 17 Add manual server connection UI and custom text input

5) Claude

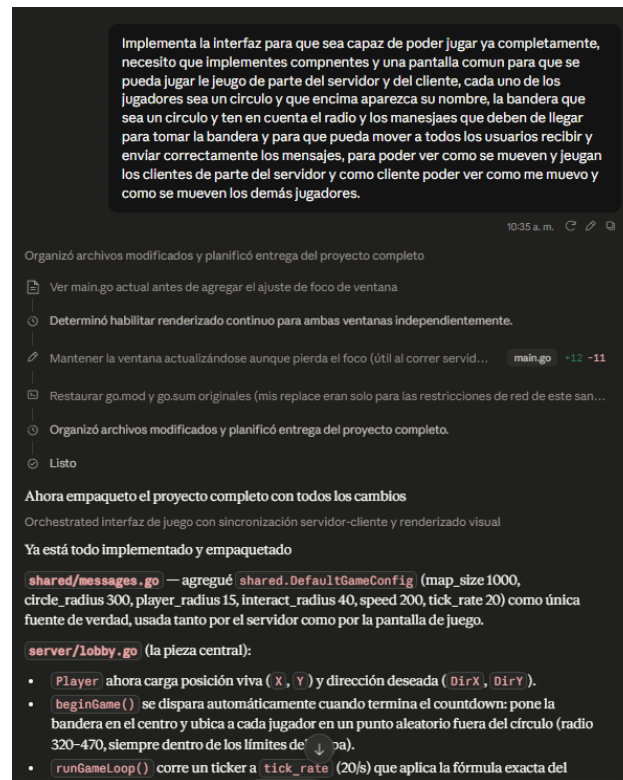


Ilustración 18 Add game interface, lobby flow, and client state updates

6) Claude

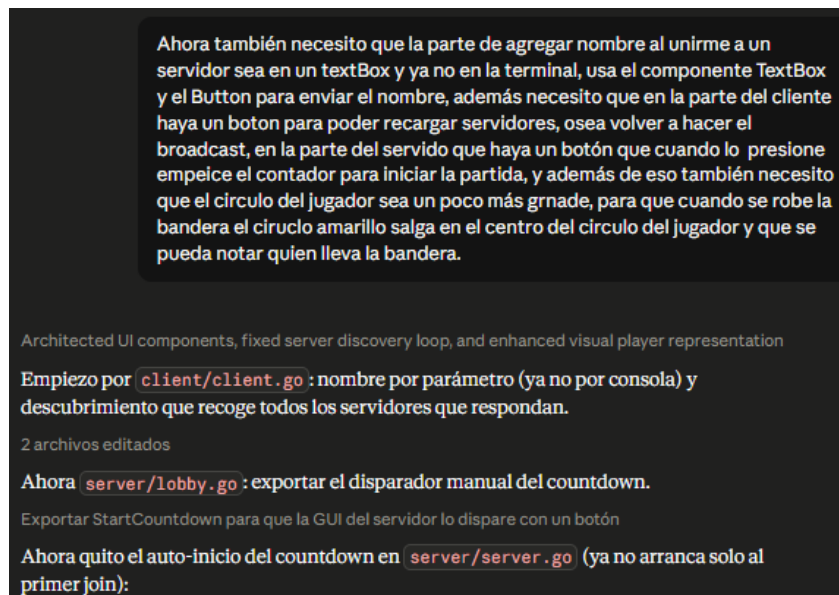


Ilustración 19 Finalize game restart flow and prevent duplicate players

7) Codex

Estándar Protocolo CTF - CC8 2026.pdf

Estándar Protocolo CTF.pdf

Revisa cuidadosamente el código del proyecto y compáralo con el estándar actualizado del protocolo CTF. Analiza primero la implementación existente y verifica que cumpla con todos los cambios introducidos en la versión más reciente del protocolo. En particular, asegúrate de implementar el descubrimiento dual por broadcast (255.255.255.255 y el broadcast de la subred), el soporte para descubrimiento manual por unicast y conexión directa mediante IP:puerto, la configuración del socket UDP con SO_REUSEADDR y SO_REUSEPORT cuando sea posible, la validación estricta de la versión del protocolo (`v == 1`), el manejo completo de errores utilizando los códigos definidos por el estándar, el límite de tamaño de mensajes de 64 KB, el procesamiento correcto del framing TCP utilizando un JSON por línea, la validación completa del mensaje `join`, el rechazo de nuevos jugadores durante `countdown` y `playing`, la máquina de estados completa de la partida, la cancelación del countdown cuando haya menos de dos jugadores, el regreso automático al lobby después de `game_over`, el manejo de todas las desconexiones según el estándar, el regreso de la bandera al centro cuando el portador se desconecte, el sistema de spawn dentro del anillo definido, la normalización del movimiento diagonal, las reglas completas de captura, robo y victoria, el procesamiento de mensajes en orden de llegada TCP como único criterio de desempate, el modelo completamente autoritativo del servidor, la coalescencia únicamente de los mensajes `state`, la estructura exacta de todos los mensajes definidos por el protocolo y las constantes obligatorias enviadas en `welcome.config`. Realiza únicamente los cambios necesarios para cumplir la especificación, sin refactorizar ni modificar código que ya funcione correctamente. Al finalizar, proporciona un resumen de los cambios implementados, los archivos modificados, la razón de cada modificación y los requisitos que ya estaban correctamente implementados. El objetivo es dejar el proyecto completamente compatible con la versión 1.2.0 del estándar del protocolo CTF.

Mostrar menos ^

Ilustración 20 align CTF protocol implementation with v1.2.0